Sprint 4 Deliverable

University Inventory Management System

Srinidh & Jithender

Sprint Number:	Sprint 4
Sprint Duration:	2 Weeks (Week 7 – Week 8)
Sprint Goal:	Implement the QR Verification module — Verification Officers scan QR codes, update item status, and every event is logged in VER_HISTORY for auditing.
Team:	Srinidh, Jithender
Course:	Software Engineering Lab
Date:	April 2026

1. Sprint Plan & Backlog

1.1 Sprint Goal

Build the core verification workflow. A Verification Officer (VO) opens the scan page, activates the device camera, scans an item's QR label, and the system decodes the QR, fetches item details, and allows the VO to mark the item as Verified. Each event is stored in the VERIFICATION table and mirrored to VER_HISTORY for auditing. This sprint covers UC-007 (Scan QR Code) and UC-008 (Update Verification Status).

1.2 Sprint Backlog

The following user stories and tasks are selected from the product backlog for Sprint 4.

ID	User Story / Task	Priority	Est. Hrs	Status
US-16	As a VO, I want to open a scan page that activates my device camera for QR scanning.	High	3	To Do
US-17	As a VO, I want the system to decode the QR and display item details automatically.	High	4	To Do
US-18	As a VO, I want to mark an item as Verified with a single button click.	High	3	To Do
US-19	As a VO, I want to see the last verification date for an item on the scan result screen.	Medium	2	To Do
US-20	As an Admin, I want all verification events logged in VER_HISTORY with officer, date, and notes.	High	3	To Do
US-21	As a VO, I want to manually enter an item ID if the camera scan fails.	Low	2	To Do
T-18	Integrate html5-qrcode library for in-browser camera scanning.	High	4	To Do
T-19	Implement POST /api/verify endpoint; writes to VERIFICATION and VER_HISTORY.	High	5	To Do
T-20	Build scan UI page: camera feed, decoded item card, Verify button.	High	4	To Do

T-21	Restrict scan/verify routes to VO role via RBAC middleware.	High	2	To Do
T-22	Write unit and integration tests for the verification flow.	Medium	3	To Do

1.3 Sprint Timeline

Day	Activity
Day 1–2	Evaluate html5-qrcode library; set up camera permission handling
Day 3–4	Implement POST /api/verify with dual-write to VERIFICATION and VER_HISTORY
Day 5–6	Build scan UI page: camera feed, QR decode, item detail card
Day 7–8	Connect UI to verify endpoint; show success/failure toast feedback
Day 9	Add manual item-ID fallback input for camera failure scenarios
Day 10–11	Restrict verify routes to VO role; verify RBAC enforcement
Day 12–13	Integration tests for the full scan-to-verify flow
Day 14	Bug fixes, sprint review and retrospective

1.4 Definition of Done

- VO can open the scan page and activate the device camera in supported browsers (Chrome, Firefox).
- Scanning a valid QR displays item name, room, type, and last verification date within 3 seconds.
- VO can click Verify; system writes one row to VERIFICATION and one to VER_HISTORY atomically.
- Scanning an unknown QR code displays a clear 'Item not found' error message.
- Routes /scan and /verify are blocked for Admin and unauthenticated users (403 returned).
- Manual item-ID lookup works as a fallback when the camera is unavailable.
- All unit and integration tests pass.
- Code committed to the repository with meaningful commit messages.

1.5 Sprint Risks

Risk	Likelihood	Impact	Mitigation
Camera API not available on all browsers or older devices.	Medium	High	Show manual entry fallback; test on Chrome Android and Safari iOS.
QR decode false positives from blurry or partial scans.	Low	Medium	Validate decoded item_id format before querying the database.
HTTPS required for camera access on mobile browsers.	Medium	High	Deploy dev server with self-signed cert or use ngrok for mobile testing.
Concurrent scans of the same item by two officers.	Low	Low	Both writes succeed; VER_HISTORY stores both with distinct timestamps.

2. Module Design

2.1 Use Cases Addressed

Use Case	Description
UC-007	Verification Officer scans a QR code and retrieves item details.
UC-008	Verification Officer marks an item as Verified; system logs the event.

2.2 Verification Flow

- 1. VO navigates to /scan — browser requests camera permission via navigator.mediaDevices.getUserMedia.
- 2. html5-qrcode decodes the QR payload (JSON containing item_id and item name).
- 3. Frontend calls GET /api/items/:id to fetch full item details from the server.
- 4. Item card is rendered showing name, room, type, and last verification date.
- 5. VO clicks Mark as Verified — frontend calls POST /api/verify with item_id.
- 6. Backend writes to VERIFICATION (status = Verified) and VER_HISTORY (audit row) in one transaction.
- 7. UI shows a green success toast; the item card updates to show the new last-verified date.

2.3 POST /api/verify — Endpoint Logic

```
router.post('/verify', authMiddleware, voOnly, async (req, res) => {
const { item_id, notes } = req.body;
const officer_id = req.user.user_id;
const conn = await db.getConnection();
await conn.beginTransaction();
try {
const [ver] = await conn.query(
'INSERT INTO VERIFICATION (item_id, officer_id, status) VALUES (?, ?, ?)',
[item_id, officer_id, 'Verified']
);
await conn.query(
'INSERT INTO VER_HISTORY (ver_id, item_id, officer_id, ver_date, notes)
VALUES (?, ?, ?, NOW(), ?)',
[ver.insertId, item_id, officer_id, notes || null]
);
await conn.commit();
res.json({ message: 'Item verified', ver_id: ver.insertId });
} catch (err) { await conn.rollback(); throw err; }
finally { conn.release(); }
});
```

3. Unit & Integration Test Summary

Test ID	Description
UT-4-01	POST /verify with valid item_id writes rows to both VERIFICATION and VER_HISTORY.
UT-4-02	POST /verify with an unknown item_id returns 404 Not Found.
UT-4-03	POST /verify called with Admin role returns 403 Forbidden.
IT-4-01	Full flow: decode QR → fetch item → post verify → assert DB rows (Mocha).
IT-4-02	Transaction rollback: if VER_HISTORY insert fails, VERIFICATION row is rolled back.

4. Sprint Review & Retrospective

4.1 Completed Stories

- US-16 through US-21 — All verification user stories completed.
- T-18 through T-22 — Library integration, API, UI, RBAC, and tests completed.

4.2 What Went Well

- html5-qrcode integration was straightforward; decode speed under 1 second in good lighting.
- Dual-write transaction pattern from Sprint 3 reused with only minor modifications.

4.3 What to Improve

- HTTPS setup for mobile camera testing took unplanned time — add to sprint checklist.
- Manual fallback input was added late; plan fallback UX upfront from the start of the sprint.